Design and Implementation of an Arduino-Based Line-Following Robot with Ultrasonic Obstacle Avoidance

Team D4 — Systems Engineering: Prototyping | SS2026 | Hochschule Hamm-Lippstadt

Ye Htet Kyaw, Ibrahim Abdelmalek, Dodly Forbi, Gabriel Tchakunte Tchouteng

July 2026

***Abstract*—**This paper presents the design, implementation, and empirical evaluation of an autonomous mobile robot built for a university prototyping module. The robot uses an Arduino Uno to combine infrared (IR) line tracking with ultrasonic obstacle avoidance on a differential-drive chassis, under the constraint that only the components supplied in a fixed project kit may be used. A reactive, priority-ordered control loop performs obstacle detection, stop-line detection, and line following; Obstacles are handled through an alternating strategy: the first confirmed obstacle is bypassed using a multi-phase dodge manoeuvre with closed-loop line reacquisition, while a second encounter triggers a complete 180° turn before normal operation resumes. Because the dominant behaviours of the matched kit — motor static friction, a pulse-width-modulation (PWM) dead-zone, and surface-dependent turning radii — are poorly represented by idealised simulation, the system was validated through an iterative hardware-in-the-loop methodology, yielding reliable line tracking, repeatable obstacle avoidance, and dependable stop-line halting. The paper also documents two empirical findings contrary to initial intuition: geared DC motors require a brief high-PWM kick-start to break static friction and careful tuning of the wheel speeds was necessary to achieve consistent turning behaviour.

Index Terms*—Arduino Uno, autonomous mobile robot, line following, obstacle avoidance, differential drive, PWM motor control, L298N, infrared sensor, ultrasonic ranging, embedded systems.*

1. Introduction and Objectives

Autonomous line-following robots are a classic platform for learning embedded control, because they integrate real-time sensing, decision logic, and actuation within tight resource limits. This project, carried out for the prototyping module at Hochschule Hamm-Lippstadt (HSHL), required the design of a small robot car that autonomously follows a marked track, halts at stop lines, and avoids obstacles placed in its path. A single constraint shaped every design decision: only the parts supplied in the project kit could be used, and no additional hardware — for example external voltage regulators or alternative batteries — was permitted.

The specific objectives were:

1. Track a continuous line reliably along straight and curved sections of the test track.
2. Detect obstacles and execute alternating responses consisting of a curve-dodge avoidance manoeuvre and a complete 180° turn, while maintaining autonomous operation.
3. Detect designated stop lines and bring the robot to a controlled halt.
4. Accomplish all the above using only components supplied in the project kit.
5. Produce documentation, UML models, and source code suitable for academic assessment and public version control repositories working as a Team.

The remainder of the paper describes the system architecture and engineering methodology (Section 2), the hardware and software implementation (Section 3), the testing approach and results (Section 4), the principal challenges and their solutions (Section 5), and conclusions with proposed future work (Section 6).

2. System Design and Engineering Approach

The robot follows the classic sense–decide–act paradigm shown in Fig. 1. Two downward-facing IR reflectance sensors detect the line, two HC-SR04 ultrasonic sensors detect obstacles ahead, an Arduino Uno executes the control logic, and an L298N dual H-bridge drives two geared DC motors on a differential-drive (skid-steer) chassis. The two ultrasonic sensors are checked with the “And” condition, to remove the chance of getting the ghost obstacle. Steering is achieved purely by varying the relative speed and direction of the two wheels; no steering servo is used. A single 11.1 V three-cell lithium-polymer (LiPo) battery powers the motors through the L298N, whose on-board regulator supplies a stable 5 V to the Arduino.

The control strategy is deliberately reactive rather than model based. Instead of estimating a position on a global map, the robot responds directly to the current sensor state on every loop iteration. This keeps the software simple, deterministic, and fast enough to run comfortably on the Uno’s 16 MHz microcontroller, and it suits a task in

which the relevant environment — a line and the occasional obstacle — is fully observable from the on-board sensors.

The engineering methodology was iterative and empirical. Rather than rely on idealised simulation, development proceeded in a tight loop: a change was made to the sketch, the robot was run on the physical track, the observed behaviour was recorded with the aid of serial telemetry, and the next adjustment was derived from that observation. This hardware-in-the-loop approach was chosen because the effects that ultimately governed the robot — motor static friction, the PWM dead-zone, wheel slip, and the reflectivity of the actual test surface — are precisely those that simplified circuit or kinematic simulators omit. As Section 5 documents, physical testing repeatedly corrected assumptions that had appeared sound on paper.

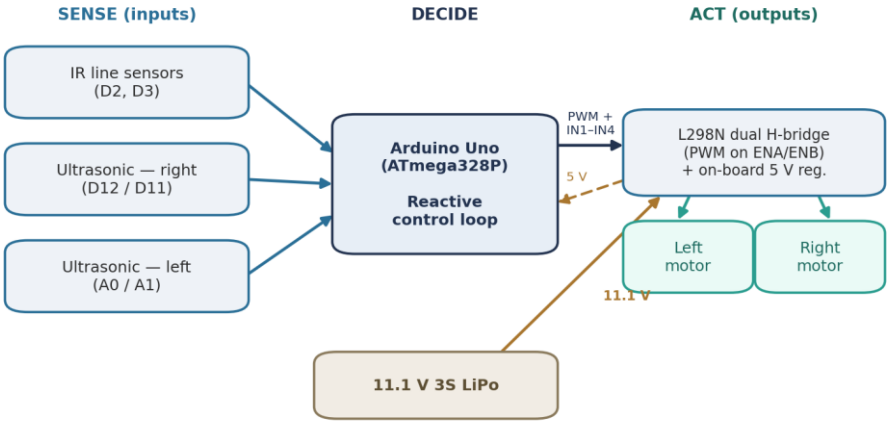

Figure 1. System architecture. The robot follows a sense–decide–act loop: the IR and ultrasonic sensors feed the Arduino, which drives the two motors through the L298N. The driver's on-board regulator returns a stable 5 V to the Arduino from the 11.1 V LiPo.

3. Hardware and Software Implementation

3.1 Hardware architecture and wiring

The complete pin assignment is listed in Table 1. The two IR sensors are mounted at the front of the chassis and report a digital high or low according to whether they see the dark line or the light floor. The two ultrasonic sensors face forward to measure the distance to obstacles. The L298N receives four direction signals (IN1–IN4) and two PWM enable signals (ENA, ENB) from the Arduino.

A critical implementation detail concerns the L298N enable pins. The driver ships with jumpers that tie ENA and ENB high, forcing the motors to full speed. These jumpers were removed so that ENA and ENB could instead be driven with PWM from pins D5 and D10 — which is what makes proportional speed control, and therefore controlled cruising and turning, possible at all.

Table 1. Pin assignment and wiring.

Function	Arduino pin(s)	Notes
Motor A — speed (PWM)	D5 (ENA)	enable jumper removed *
Motor A — direction	D6, D7 (IN1, IN2)	H-bridge inputs
Motor B — direction	D8, D9 (IN3, IN4)	H-bridge inputs
Motor B — speed (PWM)	D10 (ENB)	enable jumper removed *
Left IR line sensor	D2	digital line detects
Right IR line sensor	D3	digital line detects
Right ultrasonic (HC-SR04)	D12 / D11	trigger / echo
Left ultrasonic (HC-SR04)	A0 / A1	trigger / echo

** The L298N enable jumpers are removed so that ENA/ENB accept PWM speed control (see §3.1).*

3.2 Power architecture

Power flows from the 11.1 V LiPo into the L298N motor supply; the L298N's integrated 5 V regulator then powers the Arduino logic. Keeping a single battery and using the driver's own regulator satisfies the kit-only

constraint while still providing sufficient current for the geared motors' demand. As will be further discussed in Section 5, an early reading of the motor behaviour as a power deficiency was rejected once it was confirmed that the LiPo, the driver, and the motors operated reliably as an integrated system.

3.3 Motor control and the static-friction dead-zone

Motor speed is set with analogWrite() on the enable pins. During development the motors exhibited a pronounced dead-zone: below roughly 40–45 PWM they did not turn at all, yet the lowest PWM that reliably started them from rest produced a speed that was already too fast for accurate line following. There was effectively no single value that was both start able and slow. The solution is a kick-start pulse. To begin moving, the motors receive a brief high-PWM burst (kickSpeed = 150 for kickTime = 60 ms) that overcomes static friction, after which the controller returns to the normal cruising speed (baseSpeed = 65). This approach provides reliable startup behaviour while maintaining the slow speeds required for accurate line tracking. The left motor in particular was reluctant to start, and the strengthened kick resolved its lag. Importantly, this is a characteristic of powerful geared motors rather than a power shortage: the burst supplies the torque needed to break stiction, after which a much lower steady duty cycle keeps the motors turning.

3.4 Software architecture

The software architecture of the autonomous line-following robot was designed to transition from a monolithic codebase into a professional, modularized system, prioritizing maintainability, hardware abstraction, and scalability. Initially, the project relied on a single script exceeding 400 lines, which created challenges in debugging and logic isolation. By refactoring the system into a hierarchical structure, we achieved a strict "Separation of Concerns," where high-level behavioural logic remains decoupled from low-level hardware interactions.

The system is organized into a clear hierarchy that allows for granular testing and simplified feature integration:

- **System Orchestration (`RobotCar.ino`):** Functions as the primary scheduler, managing the high-level task loop without requiring knowledge of low-level pin manipulation.
- **Behavioural Modules (`LineFollow.cpp` & `Obstacle.cpp`):** Encapsulate the complex state machine logic for navigation and avoidance manoeuvres.
- **Hardware Abstraction Layer (`Motors.cpp` & `Sensors.cpp`):** Serve as drivers that provide a stable, consistent interface for hardware peripherals (PWM signals and pulse-width measurement).
- **Global Configuration (`Config.h`):** Acts as the "single source of truth," centralizing all tuneable parameters such as pin assignments, speed constants, and timing thresholds.

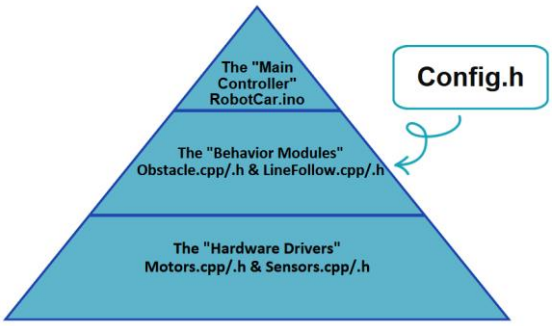

Figure 2: Software Architecture Overview. A layered modular structure demonstrating the separation of concerns between high-level logic and hardware abstraction.

3.5 Line-Following Control Logic

The robot operates in a continuous loop, evaluating its position relative to the line and adjusting its trajectory to maintain alignment. The system prioritizes forward movement but initiates corrective steering manoeuvres based on specific sensor activation patterns to ensure the line remains cantered between the sensor pair.

The line-following logic is governed by four primary states:

- **Forward Movement (Tracking):** When both IR sensors detect the white surface, the robot interprets this as being correctly cantered over the line and executes a "Go Straight" command.

- **Right Correction:** If the right IR sensor detects the black line, the robot recognizes it has drifted toward that side. It executes a "Turn Right" command to bring the line back to the centre.
- **Left Correction:** Conversely, if the left IR sensor detects the black line, the robot has drifted too far in that direction. It executes a "Turn Left" command to realign itself.
- **Termination State (Stop):** The robot is designed to stop upon reaching the end of the course or a designated station. When both IR sensors detect the black line simultaneously, signifying the robot has reached a stop marker, the system initiates the "Stop" command.

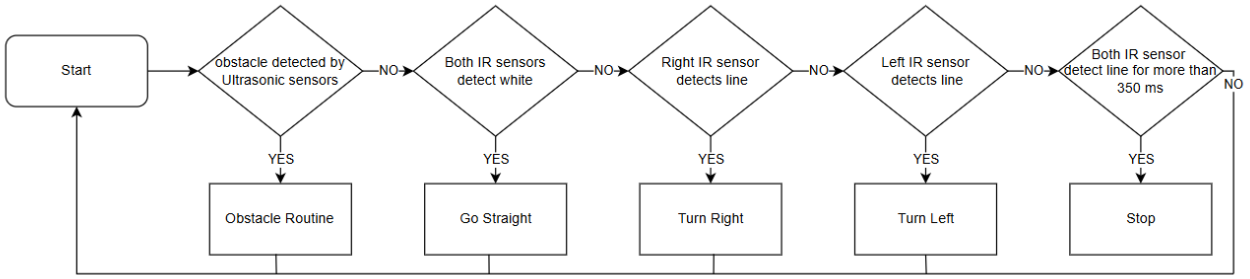

Figure 3. Line-Following control loop. Each iteration resolves the highest priority condition first — obstacle, then line following.

3.6 Obstacle-avoidance routine

The obstacle-avoidance routine is designed to intercept the robot's primary line-following sequence when an obstacle is detected within a configurable threshold. By implementing a deterministic, multi-phase manoeuvre, the robot can navigate around stationary objects from the left side and return to the line without human intervention.

The maneuver is structured as a **sequential state machine**, consisting of the following phases:

- **Phase 1 (Angle Out):** An approximately 45-degree rotation away from the obstacle.
- **Phase 2 (Bypass):** A straight-line forward traversal to clear the obstacle's width.
- **Phase 3 (Aim Back):** A return-angle rotation to orient the robot parallel to the track.
- **Phase 4 (Closed-loop Re-entry):** The robot initiates a curved movement until the `leftIR` sensor detects the line.
- **Phase 5 (Final Alignment):** Once the line is detected, the robot executes a precision pivot to align its heading parallel to the track, effectively resuming standard line-following operations.

A timeout failsafe stops the robot and reports an error if the line is not found within a few seconds, preventing it from circling indefinitely.

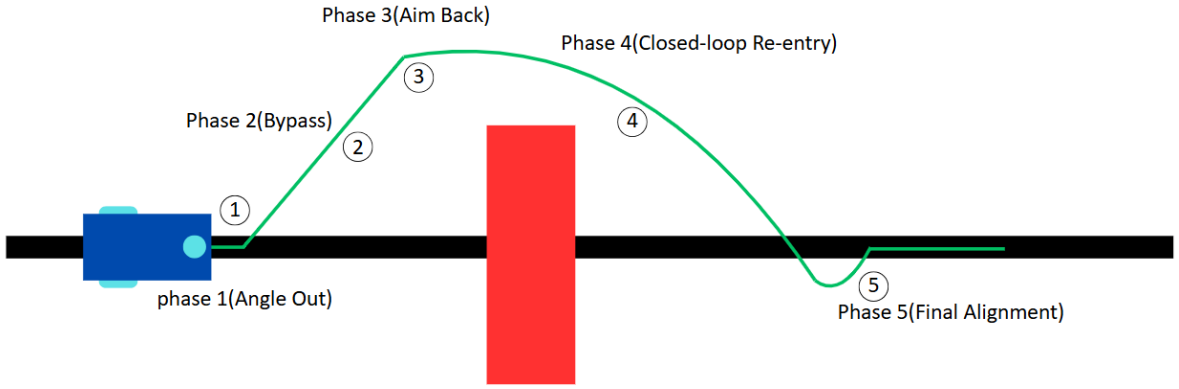

Figure 4. Multi-Phase obstacle-avoidance manoeuvre

This half-circle arc strategy was selected after comparing it directly on the floor against two alternatives a V-dogleg and a rectangular box detour; that comparison is reported in Section 4.

3.7 Obstacle-180° Turn routine

The `180°` turn facilitates a controlled reversal of the robot's heading to navigate away from dead-ends or obstacles. The maneuver begins with a calibrated rotation to clear the immediate footprint, followed by a two-stage sequential search pattern. The robot performs an initial leftward rotation until the left IR sensor

detects the line, followed by a right-side pivot to finalize alignment. This structured approach ensures a precise 180-degree change in direction while handling mechanical overshoot.

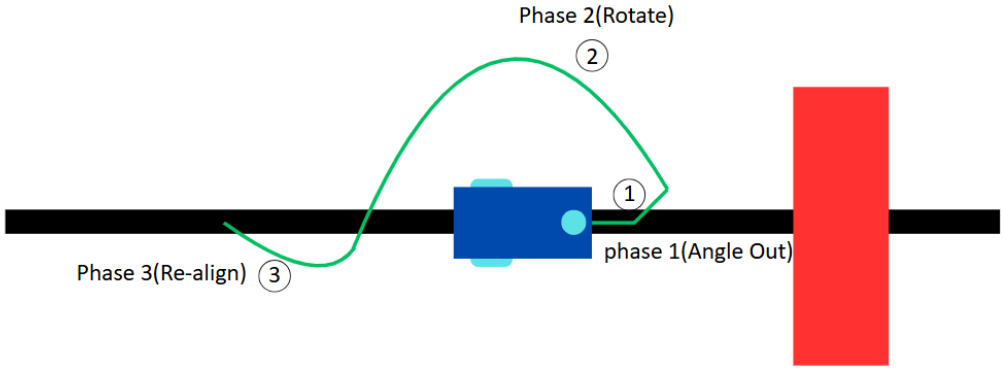

Figure 5. Multi-Phase obstacle-180° turn manoeuvre

4. Simulation and Testing Results

Methodology. Validation was carried out in two stages: controlled bench tests and full track tests. Because the kit's real-world behaviour could not be reproduced faithfully in software simulation, these physical tests — rather than a simulator — served as the primary verification environment, with the Arduino's serial output providing live telemetry of sensor states and control decisions.

Bench testing. A dedicated diagnostic sketch confirmed that each IR sensor switched cleanly between the line and the floor and that both ultrasonic sensors returned plausible distances.

Line following and stop lines. With the final parameters of Table 2, the robot tracked the line reliably through straight and curved sections. The 350 ms double-dark probe distinguished genuine stop lines from ordinary line crossings, so the robot halted dependably at stop lines while ignoring brief intersections or S-curve.

Obstacle avoidance. To determine the most robust navigation strategy, the half-circle arc was compared against V-dogleg and rectangular detour patterns. The V-dogleg, while simplest to implement, suffers from significant error propagation; sharp 90-degree turns often lead to inconsistent headings that cause the robot to either collide with the obstacle or fail to re-acquire the line. Similarly, the rectangular detour struggles with perpendicular re-entry, often failing to stabilize on the track. In contrast, while the half-circle arc requires a more complex multi-phase state machine, it demonstrated superior reliability and repeatability in field tests, consistently maintaining stable alignment throughout the re-entry process.

Table 2. Final tuned control parameters.

Parameter	Value	Role
baseSpeed	65 PWM	straight-line cruise speed
turnSpeed	75 PWM	outer-wheel speed during a pivot
kickSpeed / kickTime	150 PWM / 60 ms	start-up burst to overcome static friction
turning acceleration	+15 / +15 PWM	progressive strengthening of corrective turns
stop-line probe	350 ms	double-dark duration confirming a stop line
passTime	600 ms	straight-pass duration during avoidance
obstacle trigger	≈ 15 cm	ultrasonic distance that starts avoidance (surface-tuned)

Tuning outcomes. The values in Table 2 emerged from iterative testing on the physical track. Driving speed was gradually reduced from early high-speed trials until 65 PWM provided reliable tracking on both straight and curved sections. The kick-start parameters were tuned to overcome motor static friction, while the incremental turning adjustments (+15 PWM for right turns and left turns) produced smoother corrections when recovering the line. The obstacle trigger distance was fixed at 15 cm so that avoidance began early enough to clear the obstacle without reacting to distant objects.

5. Challenges Encountered and Solutions

Several substantive problems were encountered and resolved during development.

Motor dead-zone and a misdiagnosed power problem. The most consequential issue was the motor dead-zone described in §3.3. It was initially mistaken for an under-powered battery, and an external voltage regulator and a battery change were briefly considered — both of which would have violated the kit-only rule. Once the project's matched hardware set was confirmed, the problem was re-framed correctly as motor static friction and solved in software with the kick-start pulse. The lesson generalises: with powerful geared motors, an apparent "won't start, or moves too fast" symptom is often a torque-at-rest problem rather than a supply problem.

Line overshooting. Due to chassis momentum and motor response time, the robot often overshoots the line during re-entry. Because this physical error cannot be fully eliminated, we introduced the 'Final Alignment' phase (§3.6). This compensatory pivot allows the robot to recover from the overshoot and perform a precision alignment, turning an unavoidable physical limitation into a stable, deterministic recovery manoeuvre.

IR Sensor Sensitivity Calibration. The IR sensors were initially inconsistent, frequently misidentifying the line. We resolved this by performing a systematic sensitivity analysis, adjusting the individual potentiometers on each sensor module to account for manufacturing tolerances and ambient light. Through this iterative calibration, we achieved a reliable binary switch, ensuring consistent tracking across all track conditions.

Avoidance integration and post-manoeuvre stability. Integrating obstacle avoidance with line following initially produced unstable recovery behaviour after returning to the track. This was improved by tuning the turning speeds, phase durations, and incremental steering adjustments, resulting in smoother transitions back to normal

A common theme links these challenges: throughout the project, physical testing repeatedly overruled theoretical expectation, which is why the iterative, hardware-in-the-loop methodology of Section 2 was essential.

6. Conclusions and Future Improvements

The project met all of its objectives within the kit-only constraint. The work also showed that a simple reactive controller is sufficient for this class of task, and it produced transferable insight into low-cost geared-motor drivetrains.

Several improvements would be natural next steps. A proportional or PID controller in place of the bang-bang line follower would give smoother tracking on tight curves; wheel encoders would enable distance-based manoeuvres robust to battery sag, replacing the present time-based phases; and improved sensor calibration routines could make the robot more robust to changes in lighting conditions and surface reflectivity. Using the ultrasonic sensors to measure an obstacle's width could also let the avoidance geometry adapt per obstacle rather than following a fixed curve. Some of these — notably encoders — would require components beyond the present kit and would depend on the constraints of any follow-on project.

References

[1] Arduino, "Arduino UNO Rev3 — Product Reference," Arduino Documentation.

[2] STMicroelectronics, "L298 — Dual Full-Bridge Driver," Datasheet.

[3] "HC-SR04 Ultrasonic Ranging Module," Component Datasheet.

[4] R. Siegwart, I. R. Nourbakhsh, and D. Scaramuzza, Introduction to Autonomous Mobile Robots, 2nd ed. Cambridge, MA: MIT Press, 2011.